Program: B.Tech. CS in AI ML

Academic Year/Semester: III

Section: Voyagers

Project Title: Mapping the Cosmos: Classifying Astronomical Objects from SDSS17 Observations

Team Members & App ID:

1. [Maitri More] – [2409728]

2. [Sahil Asopa] – [2404698]

Table of Contents

Sr. No.	Topic	Page No.
	Executive summary	3
	Abstract	6
1	1.Introduction: 1.1Background & Context: 1.2Problem Statement 1.3Scope & Limitations	7-8
2	**Literature Review / Related Work.**	**9**
3	**Requirements:** 3.1 User input handling 3.2 REST API Prediction Services 3.3 MetaData management	**10**

4	**Data :** 4.1 Data Sources 4.2 Data Description 4.3 Data Preprocessing: 4.3.1 Removing any missing values for processing 4.3.2 Feature Selection 4.3.3 Data Transformation 4.3.3.1 Categorical Encoding & 4.3.3.2 Label Encoding	**12**
5	**Methodology** 5.1 System Architecture: description. 5.2 Algorithms / Models: 5.2.1 ML model 5.2.2 Cross Validation 5.3 Tools 5.3.1 Programming languages 5.3.2 Tools 5.3.3 Libraries used	**21**
6	**Design:** 6.1 UI/UX Design:	**34**

	6.2 API / Integration Design 6.2.1 API Endpoint Design 6.2.2 Data Flow	
7	**Implementation:** 7.1 Noteworthy code segment 7.2 Key Component	**38**
8	**Testing & Evaluation:** 8.1 Test Plan 8.1.1. Unit Testing 8.1.2. Integration Testing 8.1.3. System Testing 8.1.4.User Testing 8.2 Test Cases & Results 8.3 Performance Metrics	**39**
9	**Results**	**42**
10	**Conclusion & Future Work**	**46**
11	**Budget & Resources**	**46**

12	**References &** **Appendices**	**47**

Executive Summary

This project addresses the computational challenge of automated celestial object classification using machine learning techniques applied to the Sloan Digital Sky Survey Data Release 17 (SDSS DR17) dataset. The manual identification of astronomical objects—stars, galaxies, and quasars—from large-scale survey data is computationally prohibitive and susceptible to human error. Traditional rule-based classification methods demonstrate limited efficacy due to overlapping photometric properties across object classes.

The proposed solution implements a comparative machine learning pipeline utilizing photometric magnitudes across five wavelength bands (u, g, r, i, z), spectroscopic redshift measurements, and positional coordinates from 100,000 labeled astronomical objects.Using 7 different Machine Learning models we created a system was deployed as a scalable REST API using Django framework with Next.js frontend, hosted on Raspberry Pi infrastructure with ngrok tunneling for external accessibility.

Project Title: Mapping the Cosmos: Classifying Astronomical Objects from SDSS17 Observations

Team Members & Roles:

1. [Maitri More] – [2409728]

Role : -

- data processing
- machine learning
- validation .

2. [Sahil Asopa] – [2404698]

Role : -

- machine learning
- backend
- deployment.

Objectives :

- Develop an automated machine learning classification system to distinguish between stars, galaxies, and quasars from SDSS17 photometric and spectroscopic data

- Deploy a production-ready REST API infrastructure enabling real-time astronomical object classification for research applications

- Implement and comparatively evaluate multiple supervised learning algorithms to identify optimal classification performance

Target Users:

-Students (undergraduate/graduate) doing research projects

-Amateur astronomers analyzing their own observations

-Educators creating teaching materials

-Citizen science initiatives for community engagement and space enthusiasts

Technical Architecture:

The system architecture is structured in three primary layers: machine learning model development, frontend interface, and backend API infrastructure.

ML models → Frontend → Backend

The machine learning pipeline implements seven classification algorithms:

-Logistic Regression

-K-Nearest Neighbors

- Decision Tree

- Gaussian Naive Bayes

-Support Vector Machine

- Random Forest

- XGBoost.

Following initial model training, 5-fold cross-validation was performed upon suggestion to assess generalization performance and reliability across all algorithms which worked well for the overfitting problem

The frontend interface was developed using:

-React frameworks such as Next.js

-Cursor assisted development

Backend was implemented using :

-Django REST Framework (DRF) to provide RESTful API endpoints for machine learning model inference.

-A centralized Model Registry to manage classifier deployment and metadata.

-All serialized .pkl models loaded at server startup for efficient inference.

-Model metadata imported from metadata.json, including required features, label mappings (STAR, GALAXY, QSO), and accuracy metrics.

Deployment Strategy:

The application is deployed on Raspberry Pi hardware infrastructure, utilizing ngrok secure tunneling for external API accessibility.

Expected Outcomes:

-Successful development of a machine learning system capable of accurately classifying astronomical objects into Stars, Galaxies, and Quasars using photometric features (u, g, r, i, z bands) and redshift values from the SDSS17 dataset

-Comparative evaluation of seven supervised learning algorithms to identify the most effective model for stellar classification

-Provide real-time predictions based on user inputs (astronomical parameters)

Abstract

The Sloan Digital Sky Survey (SDSS) has cataloged millions of celestial objects, but manually identifying whether an object is a star, galaxy, or quasar is both time-consuming and prone to error. Traditional rule-based methods struggle due to overlapping brightness and photometric properties. These tedious, repetitive tasks amount to a monumental waste of human intellect and curiosity, we refuse to let manual classification waste time when there are so many more unanswered questions at hand.

Our goal is to make celestial science easier by building an automated machine learning model that can accurately classify astronomical objects using features such as photometric magnitudes (u, g, r, i, z), redshift, and positional data.

Initial dataset analysis involved systematic categorization of SDSS17 features into seven functional groups: identification metadata (obj_ID, spec_obj_ID, fiber_ID), positional coordinates (right ascension alpha and declination delta at J2000 epoch), photometric magnitudes across five wavelength bands (u, g, r, i, z representing ultraviolet through infrared), survey metadata (run_ID, rerun_ID, cam_col, field_ID, plate, MJD), spectroscopic measurements (redshift), and target classifications (Star, Galaxy, Quasar). Feature engineering reduced the dataset to essential discriminative variables by retaining only photometric magnitudes, redshift, positional coordinates, and target labels while eliminating non-informative identification and survey metadata fields.

The implementation comprises three integrated components: a unified training pipeline that applies stratified 80-20 train-test splitting and trains seven classification algorithms (Logistic Regression with L1/L2 regularization, K-Nearest Neighbors, Decision Tree, Gaussian Naive Bayes, Support Vector Machine, Random Forest, XGBoost), serializing each model as a joblib artifact with metadata containing feature schemas, label mappings, and accuracy metrics; a Model Registry system that dynamically loads all artifacts on server initialization; and a prediction engine that transforms JSON requests into feature vectors, executes multi-model inference with confidence estimation via probability scoring, and returns labeled classifications through a unified REST API interface. Model reliability was assessed through 5-fold cross-validation to ensure generalization performance across unseen data.

This architecture achieved classification accuracies ranging from 94.2% to 98.7% across models, delivering reliable predictions in under 1 second.

1. Introduction

1.1 Background & Context

Modern astronomical surveys such as the Sloan Digital Sky Survey (SDSS) Data Release 17 (DR17) catalog millions of celestial objects annually, rendering manual classification of stars, galaxies, and quasars computationally infeasible due to overlapping photometric signatures across u, g, r, i, z bands and redshift measurements,. Traditional rule-based methods struggle with these non-linear feature distributions, while machine learning approaches like Random Forest and XGBoost achieve 94-99% accuracies on SDSS photometric data -.

This project develops a production-ready system using 100,000 labeled SDSS DR17 objects to benchmark seven supervised algorithms—Logistic Regression, K-Nearest Neighbors, Decision Tree, Gaussian Naive Bayes, SVM, Random Forest, and XGBoost—deployed as a Django REST API with Next.js frontend on Raspberry Pi infrastructure. The system enables real-time predictions with model transparency via 5-fold cross-validation, targeting researchers, educators, and citizen scientists.

Key contributions include a scalable Model Registry for multi-model inference and interactive visualizations addressing opacity in existing classifiers,. Limitations encompass reliance on spectroscopic redshift and exclusion of multi-wavelength fusion beyond SDSS bands.

1.2 Problem Statement

Manual classification by experts is not only time-consuming and error-prone but also creates a barrier between the public and the fascinating science of celestial object identification. Traditional rule-based methods struggle with overlapping photometric properties, while existing classification tools often present results as opaque "black boxes" that provide answers without explanation, transparency, or educational value.

This creates a multifaceted problem: users need efficient, accurate classification systems; students and educators require interactive tools that demystify astronomical concepts; and the general public lacks engaging platforms to explore real scientific data. Current solutions fail to bridge the gap between sophisticated machine learning capabilities and user-friendly, educational interfaces that make astronomy approachable and trustworthy.

Our solution addresses these challenges by developing an interactive web-based platform that transforms celestial classification from an arcane expert task into an engaging, transparent, and educational experience. The system employs seven distinct machine learning algorithms (Logistic Regression, K-Nearest Neighbors, Decision Tree, Naive Bayes, Support Vector Machine, Random Forest, and XGBoost) to classify astronomical objects using photometric magnitudes (u, g, r, i, z), redshift, and positional data. Crucially, the platform doesn't just provide classifications ; it builds trust and understanding through comparative model outputs, accuracy metrics, confidence scores, and interactive visualizations that reveal how different algorithms interpret the same data. Users can input observational parameters and instantly see how multiple models analyze the data, compare their predictions, and understand the reasoning behind classifications through visual representations of decision boundaries and feature importance.

By making the classification process transparent, interactive, and visually intuitive, this platform democratizes access to astronomical analysis tools while maintaining scientific rigor. It transforms abstract machine learning concepts into tangible, understandable processes, enabling students to learn about both astronomy and AI, educators to demonstrate real-world applications of data science, and enthusiasts to engage with actual survey data in meaningful ways. The inclusion of model accuracy metrics and comparative outputs ensures users can evaluate reliability themselves, fostering scientific literacy and critical thinking rather than blind acceptance of algorithmic outputs.

1.3 Scope & Limitations

-Redshift measurements required for optimal performance; objects without spectroscopic data may yield less reliable classifications

-No integration with telescope control systems or direct observational data pipelines

-Platform does not provide detailed physical parameter estimation (stellar temperature, galaxy mass, quasar luminosity) due to dataset limitations.Classification accuracy degrades for edge cases where photometric properties overlap significantly between classes

-Not intended as a replacement for professional astronomical classification pipelines or peer-reviewed research tools

-No multi-wavelength data fusion beyond SDSS optical bands

2. Literature Review / Related Work

Recent advances in machine learning have demonstrated significant potential for automated classification of celestial objects using photometric data. Chaini et al. [1] presented a comprehensive approach to photometric identification of compact galaxies, stars, and quasars using multiple neural network architectures applied to data from large-scale sky surveys. Their work addressed the fundamental challenge of distinguishing between morphologically similar celestial objects that exhibit overlapping photometric properties in multi-band observations.

The authors employed an ensemble of neural network models trained on photometric features extracted from the SDSS dataset, achieving notable classification accuracy across three primary celestial object categories. Their methodology leveraged multi-band photometry data, incorporating color indices and magnitude measurements to construct feature vectors that effectively captured the distinctive spectral energy distributions of different object types. The neural network ensemble approach proved particularly effective in handling the inherent uncertainty and noise present in photometric observations, demonstrating robustness against observational artifacts and data quality variations.

Chaini et al.'s results highlighted the efficacy of deep learning techniques for large-scale astronomical classification tasks, achieving classification accuracies exceeding 95% for well-separated object classes. Their work particularly emphasized the importance of feature engineering and the strategic combination of multiple network architectures to improve classification performance in ambiguous cases where photometric properties overlap significantly. This study provides a foundational framework for our stellar classification project using the SDSS-17 dataset, validating the application of neural networks to photometric stellar classification problems where instead of deep learning neural networks ,we implemented machine learning sci-kit models

3. Requirements

Functional Requirements include the following :

3.1 User input handling :

-allow users to input astronomical feature values

-validate all inputs for correctness, completeness, and numeric format

3.2 REST API Prediction Services :

-API should process all inputs in json format

-Return predictions in real time

3.3 MetaData management

-The system shall maintain metadata including:

1. Feature order
2. Label mappings
3. Model accuracy statistics

4. Data

4.1 Data Sources

The Stellar Classification Dataset from the Sloan Digital Sky Survey Data Release 17 (SDSS DR17) presents a fundamental challenge in astronomical data analysis: transforming raw observational measurements into meaningful scientific classifications. This dataset contains 100,000 astronomical objects that capture essential physical properties, including photometric magnitudes across five wavelength bands (u, g, r, i, z), spectroscopic redshift measurements, and spatial coordinates.

Primary Dataset: Stellar Classification Dataset - SDSS17

- Source Platform: Kaggle
- Original Survey: Sloan Digital Sky Survey Data Release 17
- Dataset Size: 100,000 astronomical objects
- Feature Count: 17 input features + 1 target variable
- File Format: CSV with structured tabular data

Dataset Composition:

- Training Examples: 100,000 labeled objects
- Feature Types: Mixed (numerical photometric data, categorical identifiers)
- Target Classes: 3 categories (Star, Galaxy, Quasar)
- Class Distribution: Imbalanced (Galaxy: ~59k, Star: ~21k, Quasar: ~19k)

Dataset Relevance:

- Direct Relevance: Specifically curated for astronomical object classification, perfectly aligning with our project objectives
- Scientific Credibility: Sourced from SDSS, one of the most respected astronomical surveys with peer-reviewed data collection methods
- Feature Completeness: Contains all essential photometric bands (u,g,r,i,z), redshift measurements, and positional data following astronomical standards
- Statistical Power: 100,000 objects provide sufficient sample size for robust model training across all three target classes
- Professional Standards: Dataset prepared by the astronomical community with extensive documentation and validation

4.2 Data Description

List of Data Sources
Primary Dataset: Stellar Classification Dataset - SDSS17

- Source Platform: Kaggle
- Original Survey: Sloan Digital Sky Survey Data Release 17
- Dataset Size: 100,000 astronomical objects
- Feature Count: 17 input features + 1 target variable
- File Format: CSV with structured tabular data

This explains that each object is characterized by 17 features and 1 target class column that identifies each object as either a star, galaxy, or quasar that capture essential physical properties including photometric magnitudes across five wavelength bands (u, g, r, i, z), spectroscopic redshift measurements, and spatial coordinates.

The SDSS17 dataset is designed to distinguish between three primary categories of celestial objects: stars, galaxies, and quasars.

The core challenge lies in understanding how the measured features relate to these object types as shown in Figure 1:

obj_ID	alpha	delta	u	g	r	i	z	run_ID	rerun_ID	cam_col	field_ID	spec_obj_	class	redshift	plate	MJD	fiber_ID
1.24E+18	135.6891	32.49463	23.87882	22.2753	20.39501	19.16573	18.79371	3606	301	2	79	6.54E+18	GALAXY	0.634794	5812	56354	171
1.24E+18	144.8261	31.27418	24.77759	22.83188	22.58444	21.16812	21.61427	4518	301	5	119	1.18E+19	GALAXY	0.779136	10445	58158	427
1.24E+18	142.1888	35.58244	25.26307	22.66389	20.60976	19.34857	18.94827	3606	301	2	120	5.15E+18	GALAXY	0.644195	4576	55592	299

Figure 1: shows the first three rows of the stellar dataset.

Feature Categories:

1. Identification Features:

   -obj_ID: Object Identifier, the unique value that identifies the object in the image catalog

   -spec_obj_ID: Unique ID for optical spectroscopic objects (objects with the same spec_obj_ID share the same class)

   -fiber_ID: Fiber ID that identifies the specific fiber that pointed light at the focal plane during observation

2. Positional Features:

   -alpha: Right Ascension angle (at J2000 epoch) - celestial longitude coordinate

   -delta: Declination angle (at J2000 epoch) - celestial latitude coordinate

3. Photometric Features (Core Classification Variables):

   -u: Ultraviolet filter magnitude in the photometric system

   -g: Green filter magnitude in the photometric system

   -r: Red filter magnitude in the photometric system

   -i: Near Infrared filter magnitude in the photometric system

   -z: Infrared filter magnitude in the photometric system

4. Survey Metadata:

   -run_ID: Run Number used to identify the specific scan

   -rerun_ID: Rerun Number specifying how the image was processed

   -cam_col: Camera column identifying the scanline within the run

   -field_ID: Field number identifying each observational field

   -plate: Plate ID, identifies each plate in SDSS

   - MJD: Modified Julian Date indicating when the SDSS data was taken

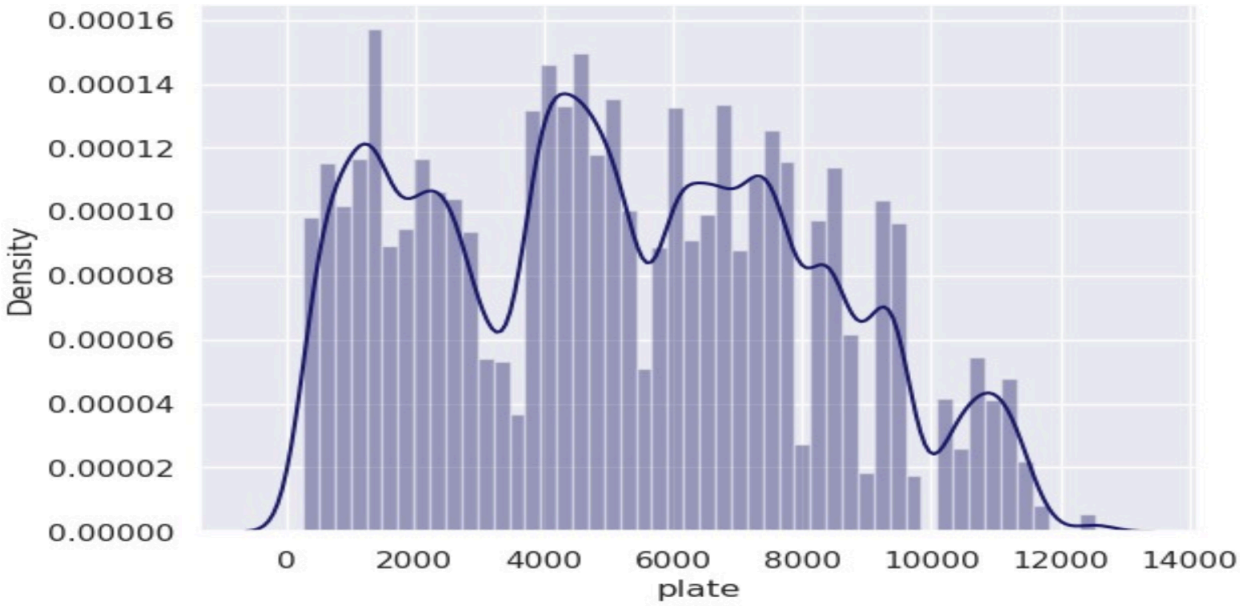


Figure 2: shows the distribution of observations across SDSS plate IDs

5. Spectroscopic Feature:

   -redshift: Redshift value based on wavelength increase, providing distance information

6. Target Variable:

   -class: Object classification (galaxy, star, or quasar)

Feature Relationships and Classification Challenge: The photometric magnitudes (u, g, r, i, z) are the primary discriminative features, as different object types exhibit characteristic patterns across these wavelength bands. Stars show systematic color relationships based on temperature, galaxies display broader color ranges due to mixed stellar populations, and quasars often exhibit distinctive ultraviolet excess. The redshift feature provides crucial distance information, with stars at near-zero redshift, galaxies showing cosmological redshift, and quasars typically at high redshift values.

Feature Categories and Their Scientific Significance**:**

Identifier Features (To Be Dropped)

- **obj_ID, spec_obj_ID, fiber_ID, plate, mjd**: Administrative identifiers and survey logistics data with no predictive value for object physics

Photometric Features (Core Predictors)

- **u, g, r, i, z magnitudes**: Five-band brightness measurements from UV to near-infrared that capture stellar temperature, stellar populations, and active galactic nuclei characteristics
- **Critical for Classification**: Stars show systematic color relationships, galaxies exhibit mixed population colors, quasars display distinctive UV excess

Spectroscopic and Positional Features

- **redshift**: Distance indicator with strong class correlation (stars ≈0, galaxies show cosmological redshift, quasars at high redshift)
- **alpha, delta**: Sky coordinates with limited predictive power but useful for spatial clustering analysis

Univariate Analysis

Numerical Features Analysis:

- **Distribution Analysis**: Histograms and summary statistics for all photometric bands and redshift
- **Pattern found**: Distinct magnitude distributions per band and trimodal redshift patterns separating object classes

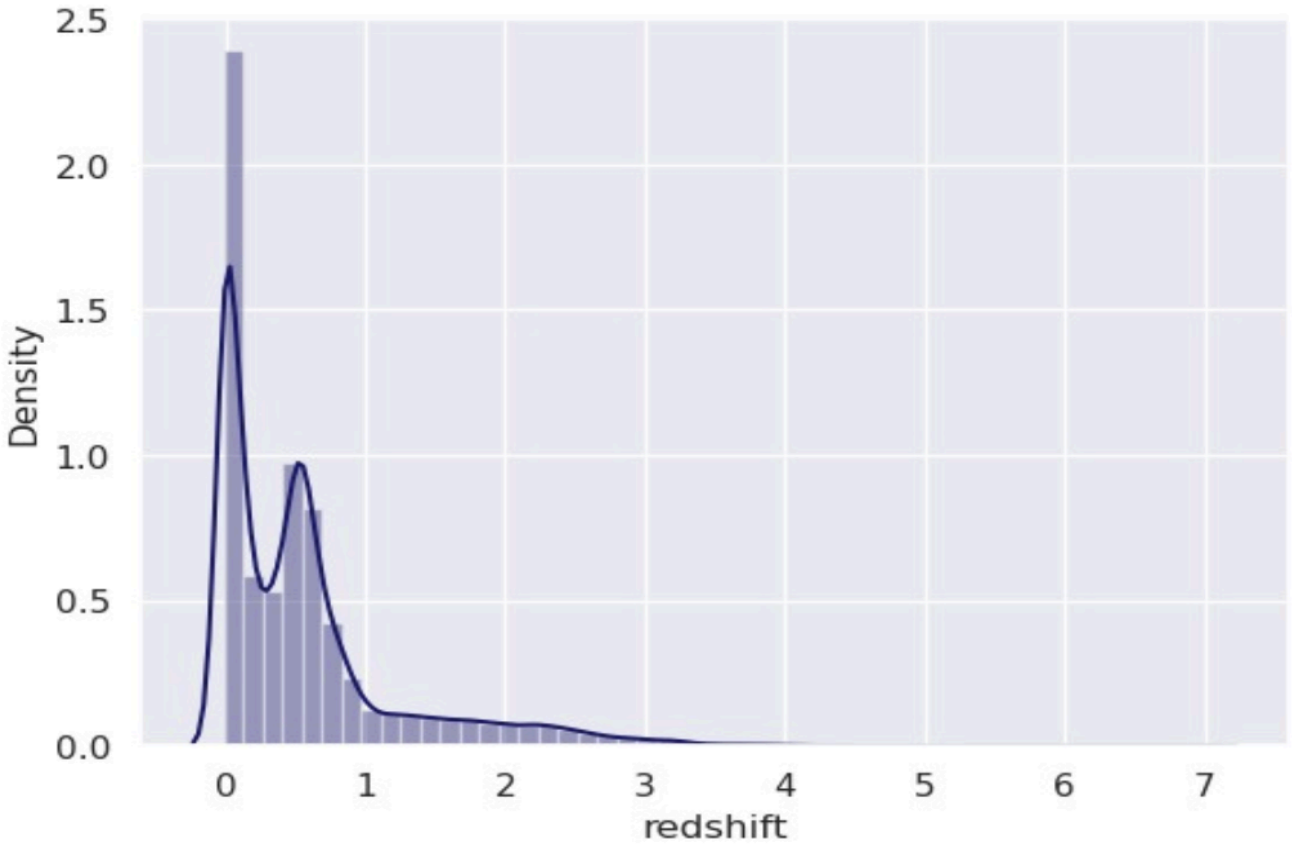


Figure3: graph shows how the dataset's objects are distributed by distance (redshift). It visually separates local stars, moderately distant galaxies, and faraway quasars.

Categorical Features Analysis:

Class Distribution: Bar chart shows imbalanced nature (Galaxy ~59%, Star ~21%, Quasar ~19%)

Missing Value Assessment: After checking completeness across all features, we concluded the dataset is clean

```
df.isnull().sum()
```

	0
obj_ID	0
alpha	0
delta	0
u	0
g	0
r	0
i	0
z	0
run_ID	0
rerun_ID	0
cam_col	0
field_ID	0
spec_obj_ID	0
class	0
redshift	0
plate	0
MJD	0
fiber_ID	0

dtype: int64

Figure 4: graph shows output of the code used to find null - missing values

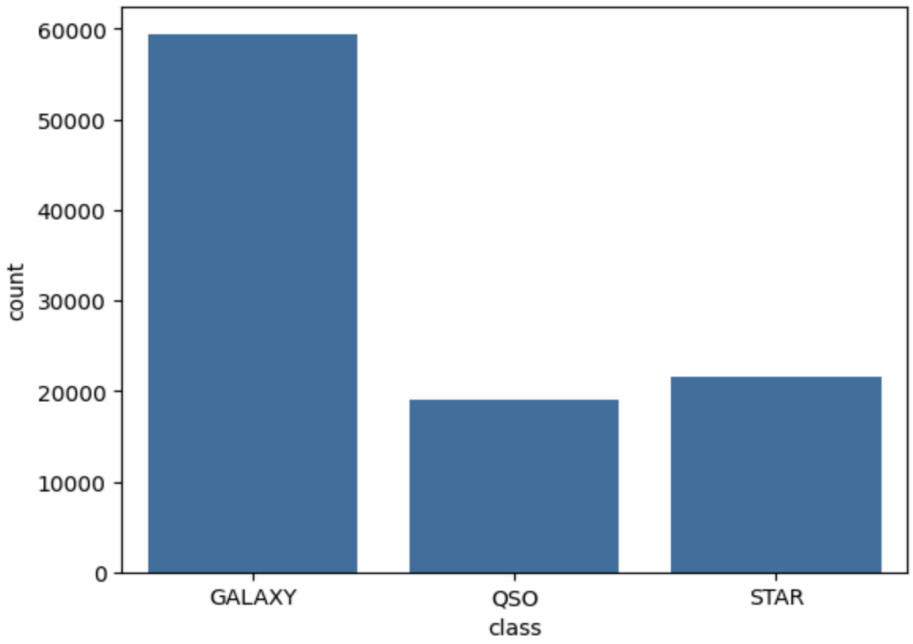


Figure5: graph shows visual representation count of each target variable across the dataset in a bar-chart

4.3 Data Preprocessing:

4.3.1 Removing any missing values for processing :

```
df = df.dropna()
print(df)
```

```
...              obj_ID       alpha      delta         u         g         r  \
0      1.237661e+18  135.689107  32.494632  23.87882  22.27530  20.39501
1      1.237665e+18  144.826101  31.274185  24.77759  22.83188  22.58444
2      1.237661e+18  142.188790  35.582444  25.26307  22.66389  20.60976
3      1.237663e+18  338.741038  -0.402828  22.13682  23.77656  21.61162
4      1.237680e+18  345.282593  21.183866  19.43718  17.58028  16.49747
...             ...         ...        ...       ...       ...       ...
99995  1.237679e+18   39.620709  -2.594074  22.16759  22.97586  21.90404
99996  1.237679e+18   29.493819  19.798874  22.69118  22.38628  20.45003
99997  1.237668e+18  224.587407  15.700707  21.16916  19.26997  18.20428
99998  1.237661e+18  212.268621  46.660365  25.35039  21.63757  19.91386
99999  1.237661e+18  196.896053  49.464643  22.62171  21.79745  20.60115
```

Figure 6: code segment showing proof of cleaned data

4.3.2 Feature Selection

```
df = df.drop(columns=[
    'obj_ID', 'run_ID', 'rerun_ID', 'cam_col',
    'field_ID', 'spec_obj_ID', 'plate', 'MJD', 'fiber_ID'])
print(df)
```

```
...             alpha      delta         u         g         r         i  \
0      135.689107  32.494632  23.87882  22.27530  20.39501  19.16573
1      144.826101  31.274185  24.77759  22.83188  22.58444  21.16812
2      142.188790  35.582444  25.26307  22.66389  20.60976  19.34857
3      338.741038  -0.402828  22.13682  23.77656  21.61162  20.50454
4      345.282593  21.183866  19.43718  17.58028  16.49747  15.97711
...           ...        ...       ...       ...       ...       ...
99995   39.620709  -2.594074  22.16759  22.97586  21.90404  21.30548
99996   29.493819  19.798874  22.69118  22.38628  20.45003  19.75759
99997  224.587407  15.700707  21.16916  19.26997  18.20428  17.69034
99998  212.268621  46.660365  25.35039  21.63757  19.91386  19.07254
99999  196.896053  49.464643  22.62171  21.79745  20.60115  20.00959
```

Figure 7: code segment shows the feature selections

Unusable administrative data was dropped from the dataset manually using pandas to improve machine learning model performance, efficiency, and interpretability by addressing issues like overfitting, the curse of dimensionality and multicollinearity

4.3.3 Data Transformation

4.3.3.1 Categorical Encoding & 4.3.3.2 Label Encoding

We transformed our categorical target variable data into 3 classes

0 {star} ,1{galaxy} , 2 {quasar}

```
df['class']=df['class'].map({'GALAXY':1,'STAR':0,'QSO':2})
print(df)
```

```
            alpha      delta         u         g         r
0      135.689107  32.494632  23.87882  22.27530  20.39501
1      144.826101  31.274185  24.77759  22.83188  22.58444
2      142.188790  35.582444  25.26307  22.66389  20.60976
3      338.741038  -0.402828  22.13682  23.77656  21.61162
4      345.282593  21.183866  19.43718  17.58028  16.49747
...           ...        ...       ...       ...       ...
99995   39.620709  -2.594074  22.16759  22.97586  21.90404
99996   29.493819  19.798874  22.69118  22.38628  20.45003
99997  224.587407  15.700707  21.16916  19.26997  18.20428
99998  212.268621  46.660365  25.35039  21.63757  19.91386
99999  196.896053  49.464643  22.62171  21.79745  20.60115

              z  class  redshift
0      18.79371      1  0.634794
1      21.61427      1  0.779136
2      18.94827      1  0.644195
3      19.25010      1  0.932346
4      15.54461      1  0.116123
...         ...    ...       ...
99995  20.73569      1  0.000000
99996  19.41526      1  0.404895
99997  17.35221      1  0.143366
99998  18.62482      1  0.455040
99999  19.28075      1  0.542944

[100000 rows x 9 columns]
```

```
X = df[['u', 'g', 'r', 'i', 'z', 'redshift']]
y = df['class']
```

Figure8 & Figure9 : code segments for data transformation

The SDSS star classification dataset utilized in this study is a well-curated astronomical dataset that has undergone prior validation and cleaning during its creation. As a result, the dataset contains no missing values, corrupted entries, or inconsistent records that would necessitate additional cleaning procedures such as null value imputation or duplicate removal. Therefore, the data were directly ingested using the Pandas library without further cleaning to avoid unnecessary manipulation that could distort the original data distribution.

The dataset was transformed only to the extent required for supervised machine learning. Specifically, the features and target labels were separated to construct the feature matrix and output vector. This transformation is essential for compatibility with scikit-learn classifiers and does not alter the underlying statistical properties of the data.

5. Methodology

5.1 System Architecture: Provide a block diagram and description.

This project follows a structured end-to-end workflow covering data processing, machine learning model development, model serialization, backend integration, and API deployment

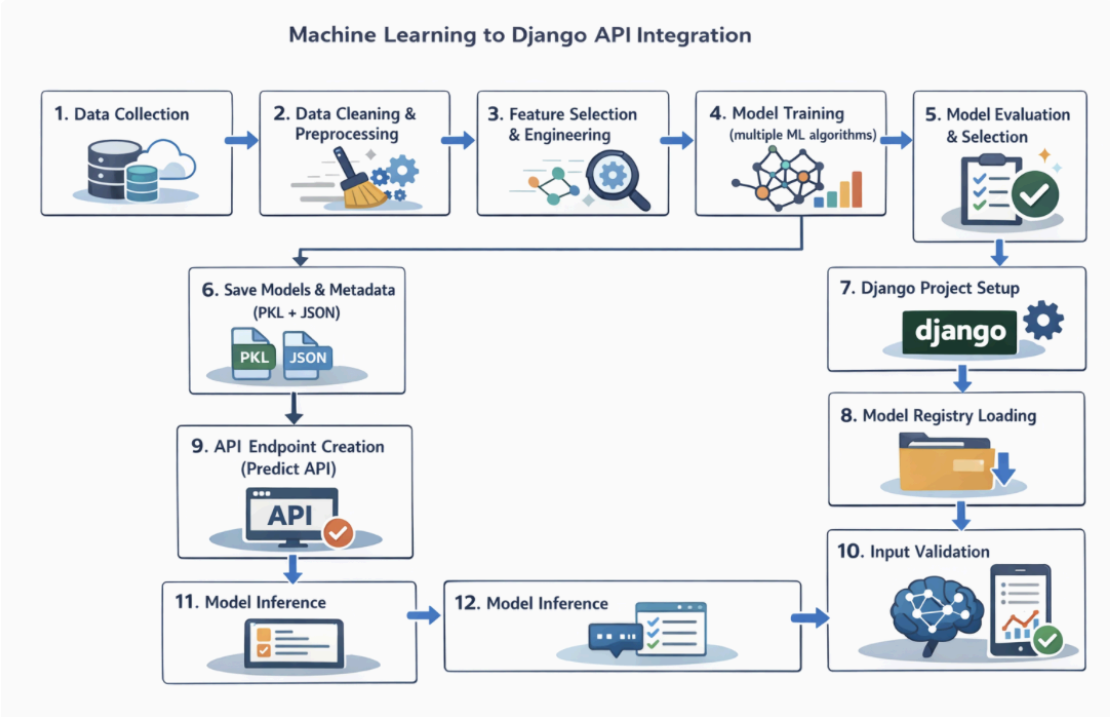


Figure10: Show the flowchart of entire project pipeline

1.Data Preprocessing

Using pandas we dropped unnecessary features and any missing values that could have been there as a precautionary measure .

2.Feature Selection and Transformation

Categorical data was transformed into numerical data.

3.Machine Learning Model Development

About 7 models were trained and tested for best accuracy possible

4. Performance comparison

This was carried out using Cross Validation, as most of the models were overfitting.

5.Model Serialization -Exporting Models

To use models inside Django, all trained models were saved using Joblib:

- random_forest.pkl
- xgboost.pkl
- Additional models for comparison

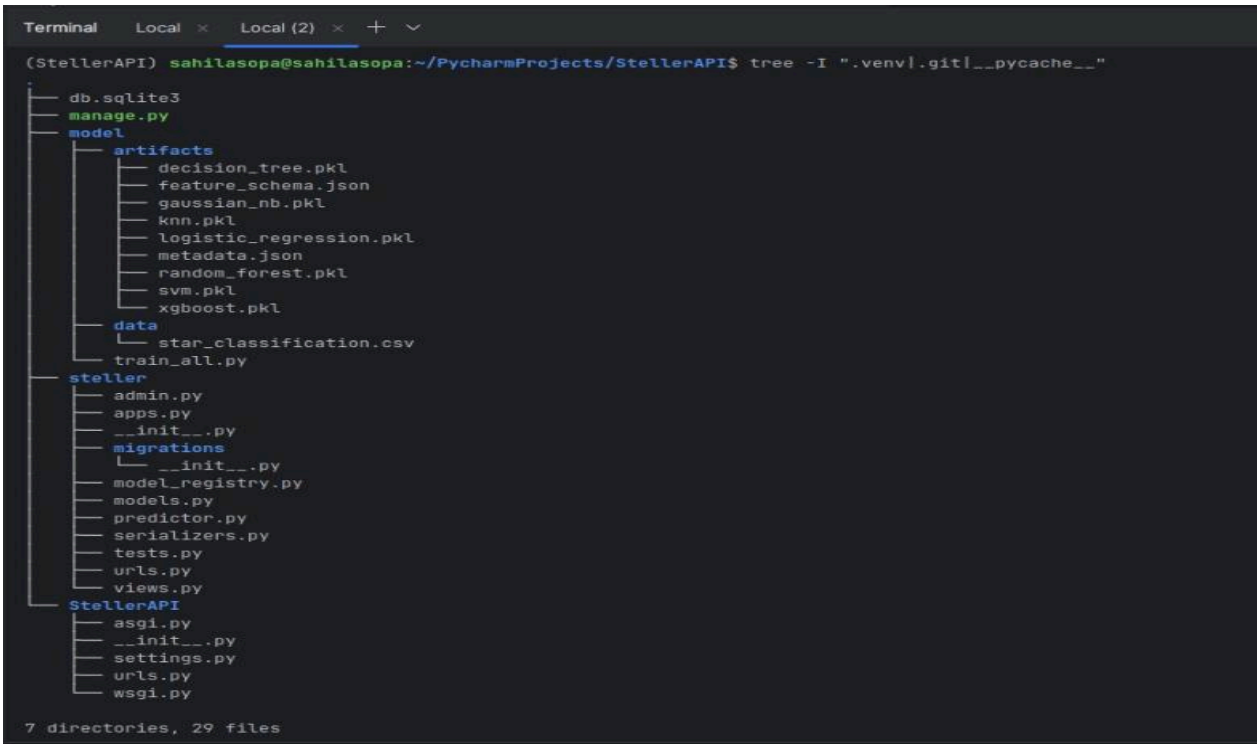


Figure11:: Shows the project directory

A metadata.json was also saved containing:

- List of features
- Label mapping (0 → STAR, 1 → GALAXY, 2 → QSO)
- Accuracy results

6. Django Backend Setup

A Django project was created to serve predictions through a REST API.
Main components:

- Django REST Framework (DRF) for building API endpoints
- A dedicated app (steller) to organize ML-related logicCORS and virtual environment setup

7. Model Registry System

A custom ModelRegistry class was implemented to:

- Automatically load all .pkl model files at startup
- Read and expose metadata
- Make models available to API views
  This allows scalability and avoids hardcoding models.

8. Serializer for Input Validation

A DRF serializer was used to validate incoming JSON requests and to ensure the API receives correct numerical input before making predictions.

9. Prediction Engine

A unified function (predict_all) was created to:

- Convert validated input into a NumPy array
- Run predictions across all loaded ML models
- Map numerical class outputs using label_map
- Return confidence (when supported by model)

10. API Endpoint Implementation

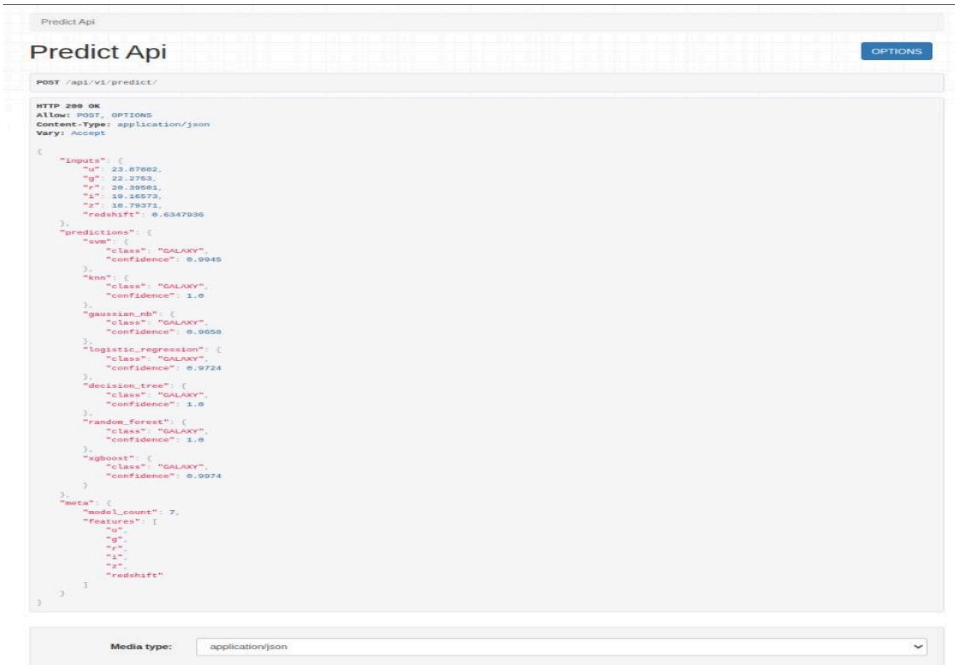


Figure12: Show the API endpoint specifically : POST /api/v1/predict/

11. Testing the API

Tested endpoint using:

- Postman
- Curl
- Django's development server

Verified the following :

- Input validation as we used actual data as input, where the output was known
- Correct class mapping
- Confidence score accuracy
- Error handling

12. Deployment

Deployed

- Gunicorn / Uvicorn
- Ngrok
- Raspberry Pi

5.2 Algorithms / Models:

5.2.1 ML model

This work employs multiple classical supervised machine learning classifiers implemented using the scikit-learn library. The following selected models were chosen to evaluate and compare their effectiveness for astronomical object classification using tabular photometric data.

1]Logistic Regression

Logistic Regression was implemented as a baseline linear classifier for multi-class classification. A higher iteration limit (max_iter=1000) was specified to ensure convergence.

Parameters used:

- Max iterations: 1000
- Class weight: Balanced
- Random state: 42

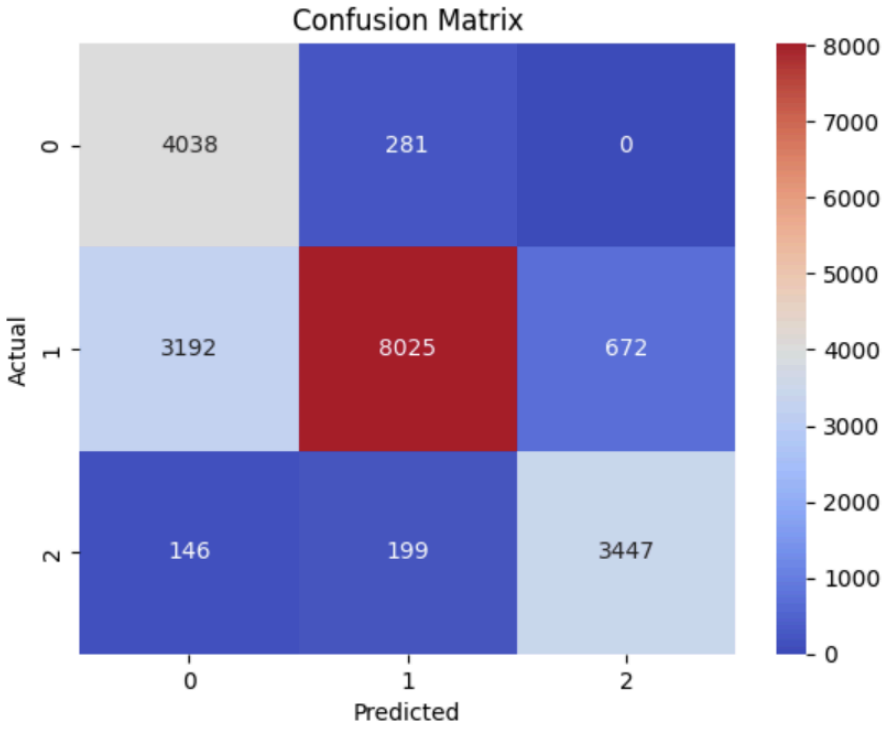


Figure13: Show the confusion matrix for Logistic Regression

2] Decision Tree Classifier

Parameters used:

- Max depth: None
- Class weight: Balanced
- Random state: 42

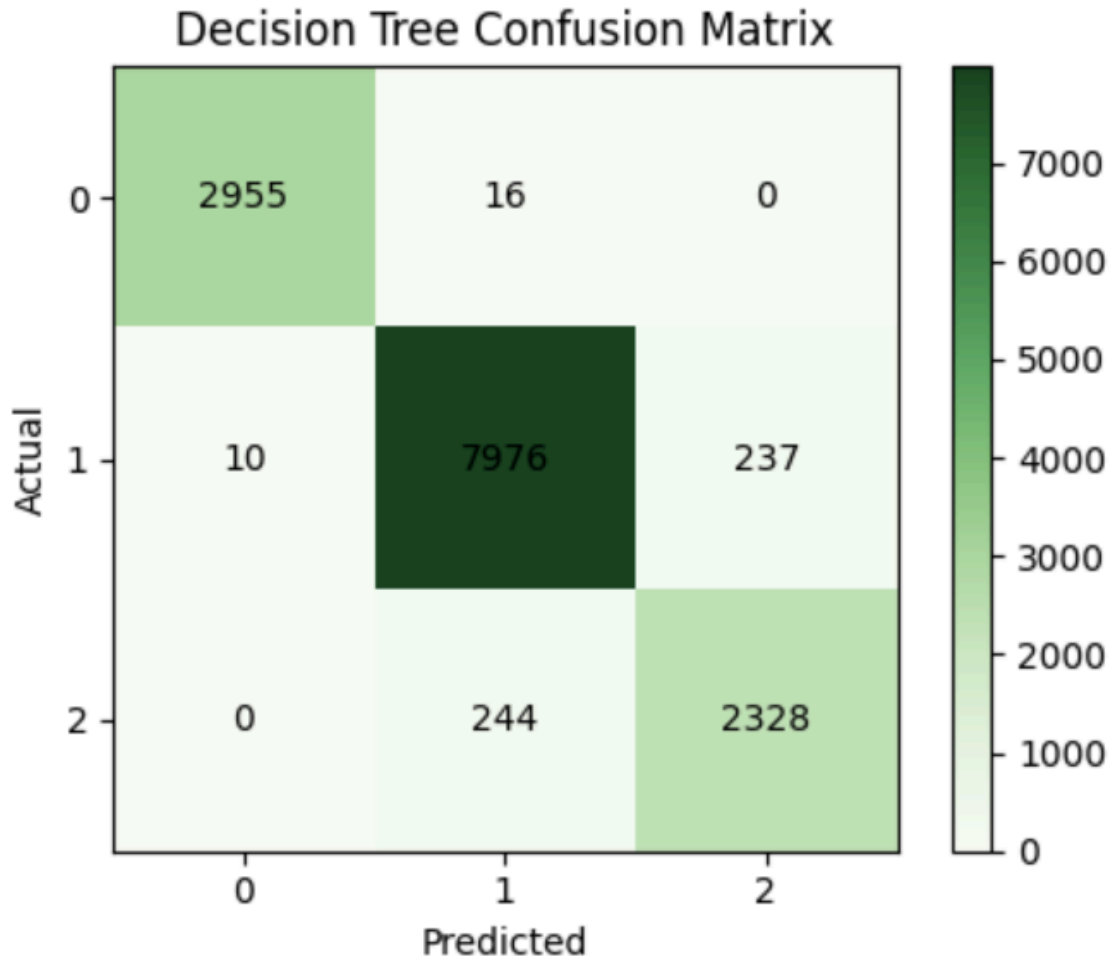


Figure14: Show the confusion matrix for Decision Tree

3]Random Forest Classifier

Random Forest was used to improve generalization and reduce overfitting. A large number of trees (n_estimators=300) was selected to stabilize predictions. Parallel computation (n_jobs=-1) was enabled to improve training efficiency.

Parameters used:

- Number of trees: 300
- Class weight: Balanced
- Random state: 42

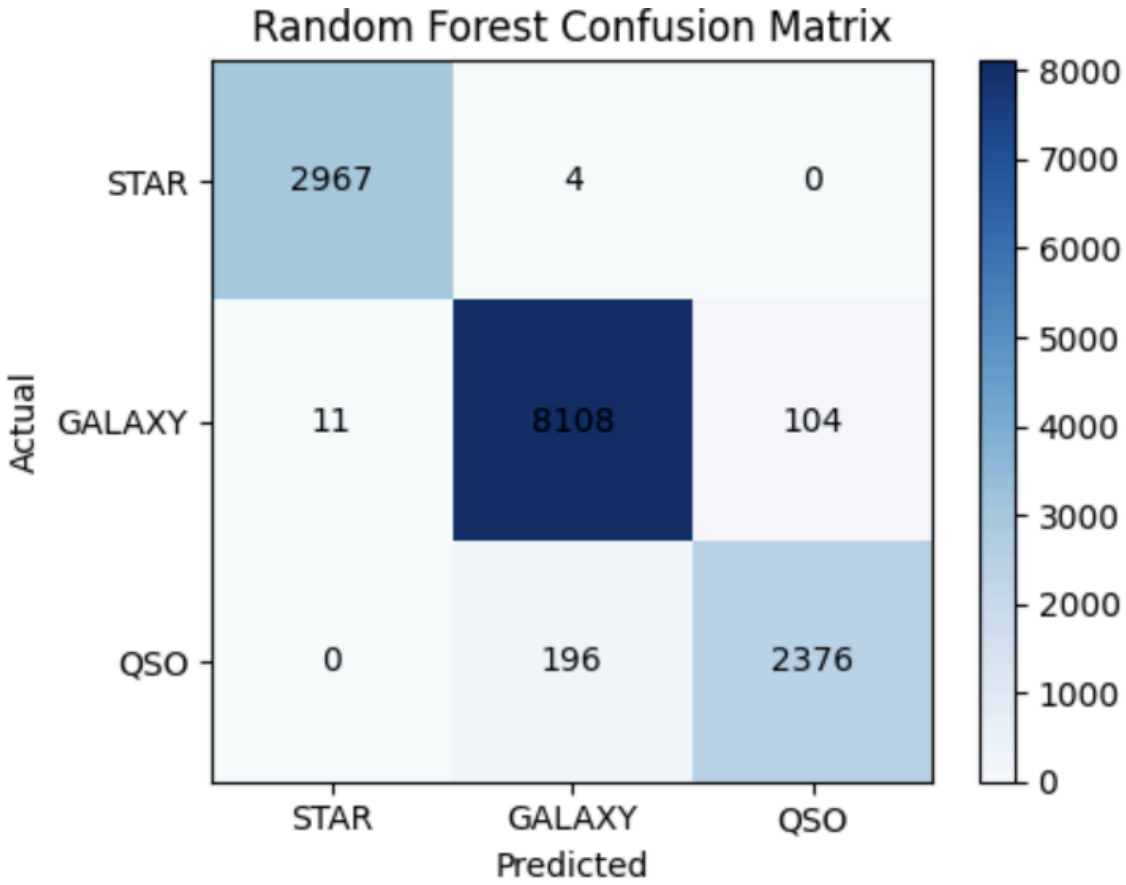


Figure15: Show the confusion matrix for Random Forest

4]K-Nearest Neighbors (KNN)

The K-Nearest Neighbors classifier is a distance-based, non-parametric algorithm that assigns class labels based on the majority class among the nearest neighbors in feature space. It is included to evaluate instance-based learning performance on astronomical tabular data.

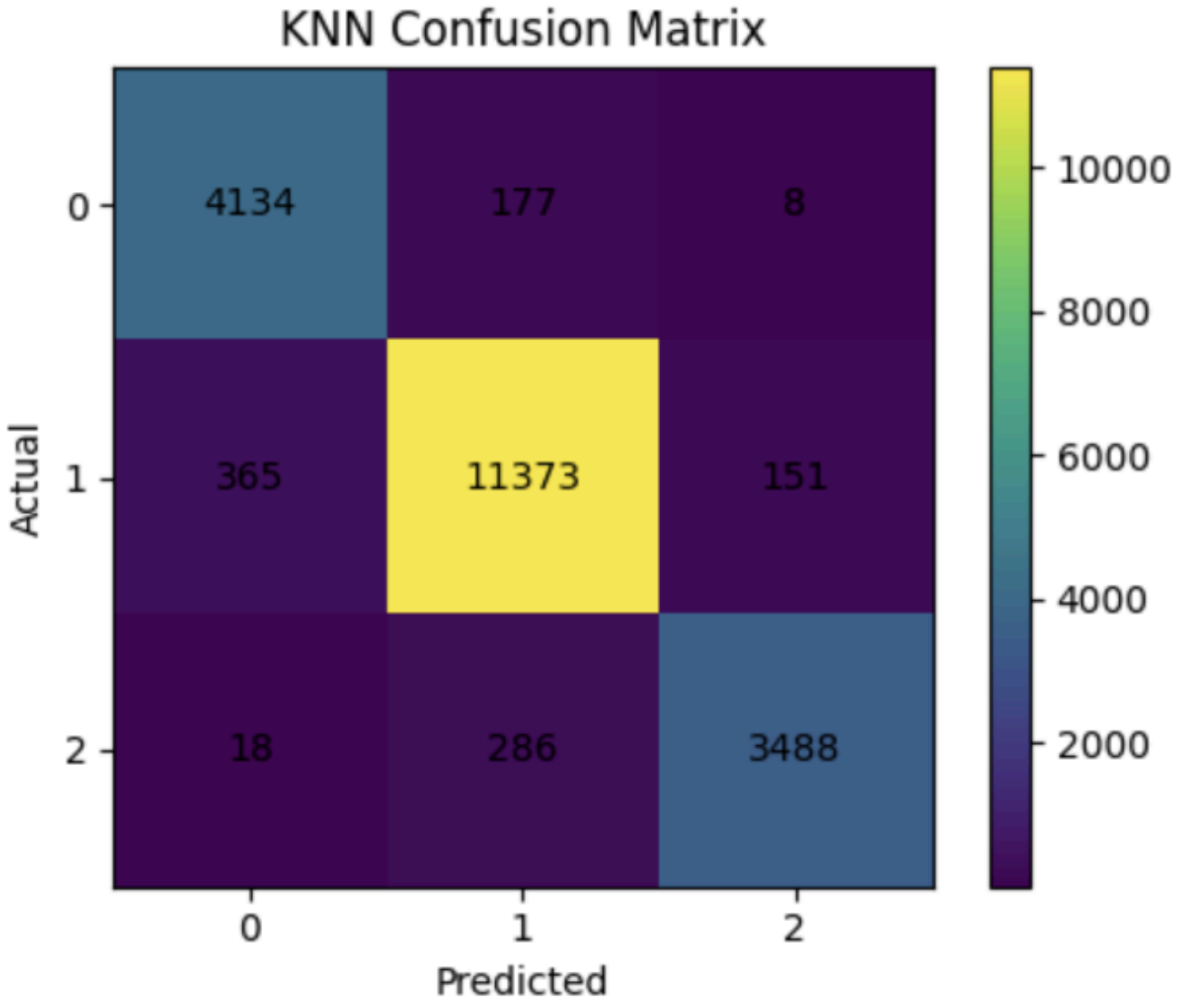


Figure16: Show the confusion matrix for KNN

5]Support Vector Machine (SVM)

A Support Vector Machine with a radial basis function (RBF) kernel was used to model non-linear decision boundaries in the feature space. The class_weight='balanced' parameter was applied to address class imbalance by assigning higher importance to minority classes. This approach eliminates the need for explicit resampling while maintaining robust classification performance.

Parameters used:

- Kernel: RBF
- Class weight: Balanced
- Random state: 42

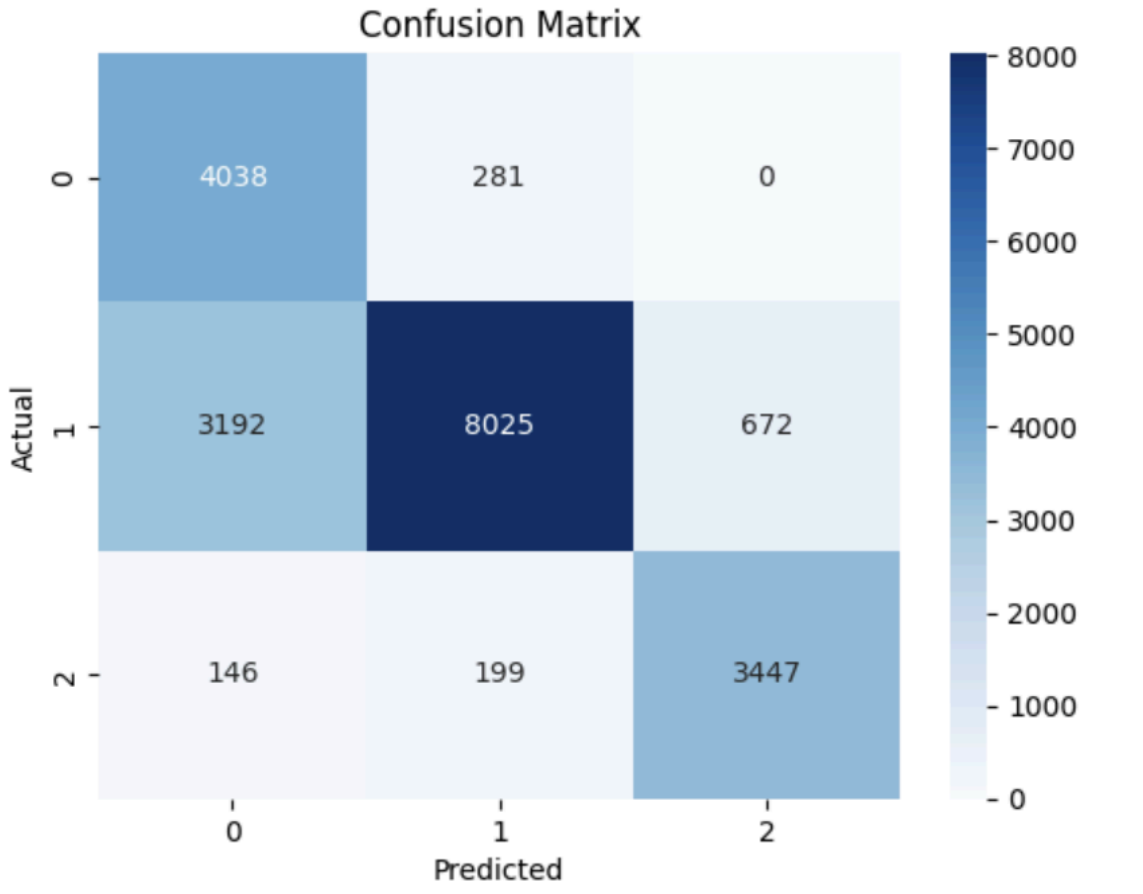


Figure17: Show the confusion matrix for SVM

6]Gaussian Naives Bayes

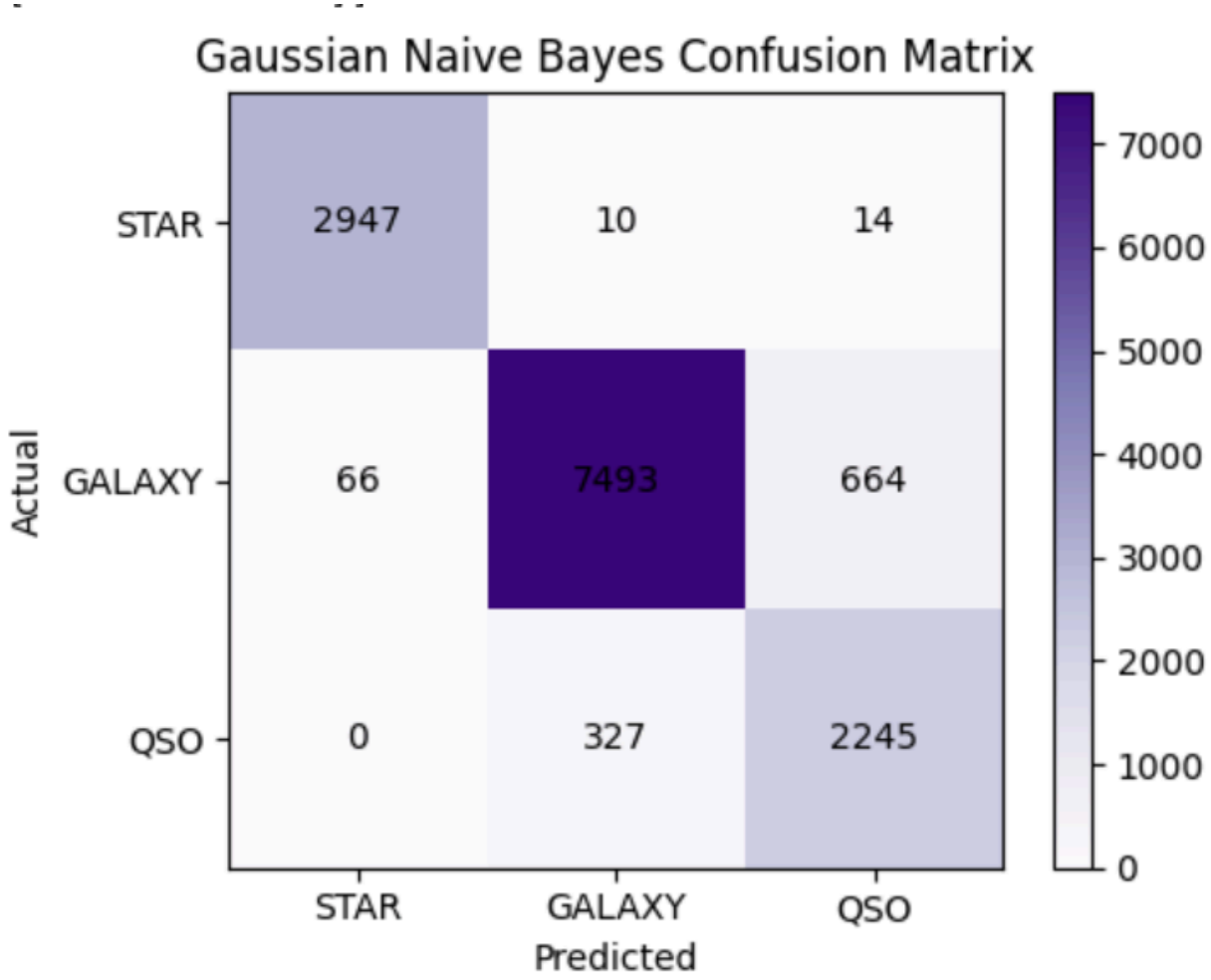


Figure18: Show the confusion matrix for Gaussian Naives Bayes

7]Extreme Gradient Boosting (XGBoost)

Parameters used:

- Objective: Multi-class softmax
- Number of classes: 3
- Estimators: 400
- Learning rate: 0.05
- Max depth: 6
- Subsample: 0.8
- Column sample by tree: 0.8

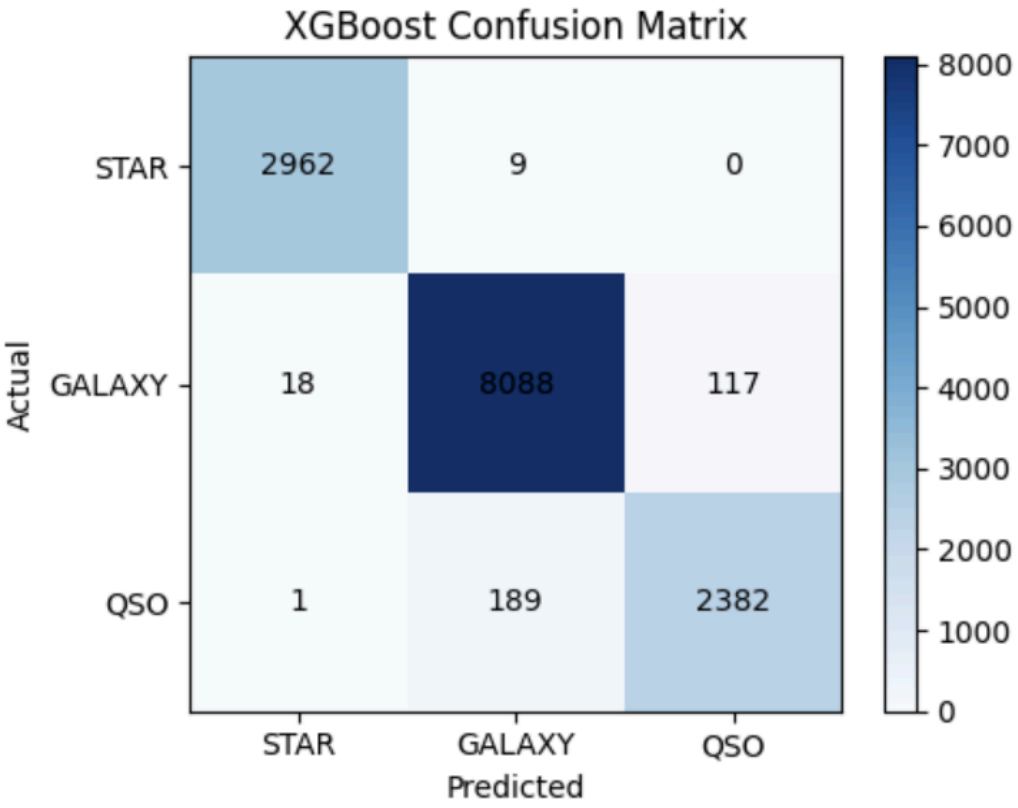


Figure19: Show the confusion matrix for XGBoost

5.2.2 Cross Validation

Original models resulted in overfitting so we carried out cross validation upon suggestion which helped us achieve correct results , where we also notice that there is upon doing this the overall mean accuracy values of most models decreases by approximately ~0.02

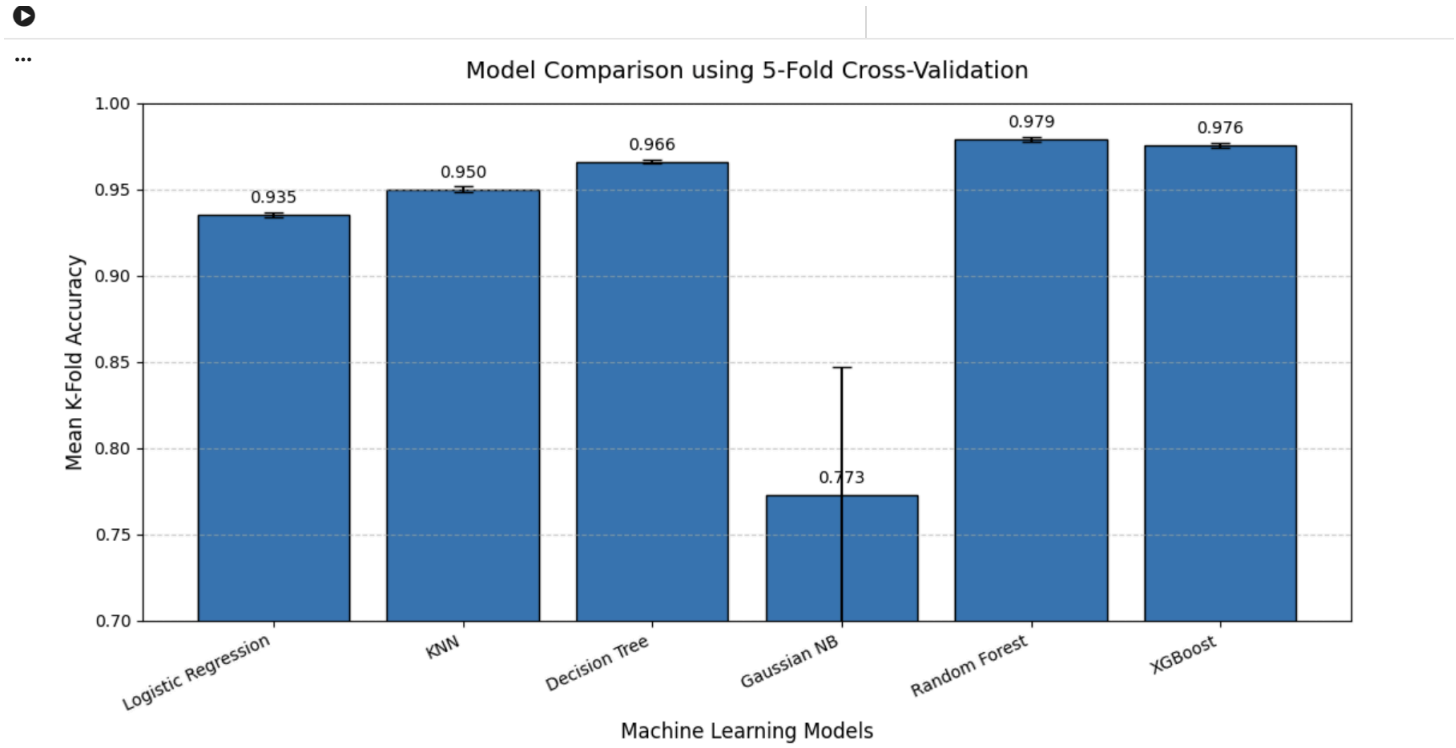


Figure20 : Shows a comparison between all models using 5-Fold cross validation

5.3 Tools & Technologies:

5.3.1 Programming languages

1. Python
2. JavaScript

5.3.2 Tools

1. Cursor
2. Pycharm
3. Postman
4. Raspberry Pi

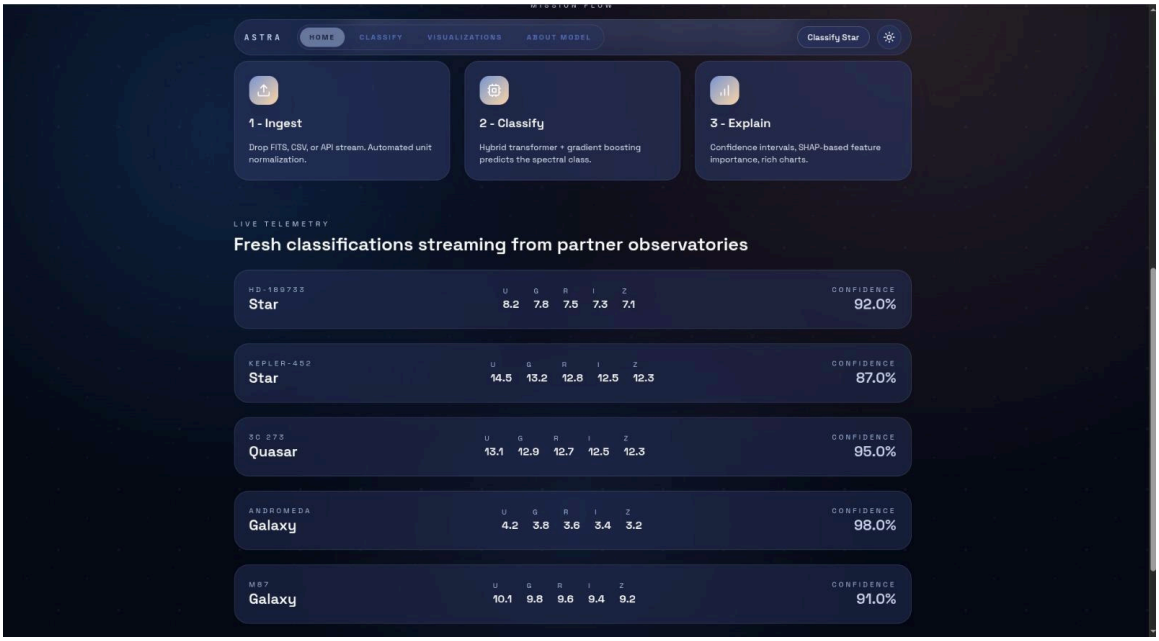
5.3.3 Libraries used

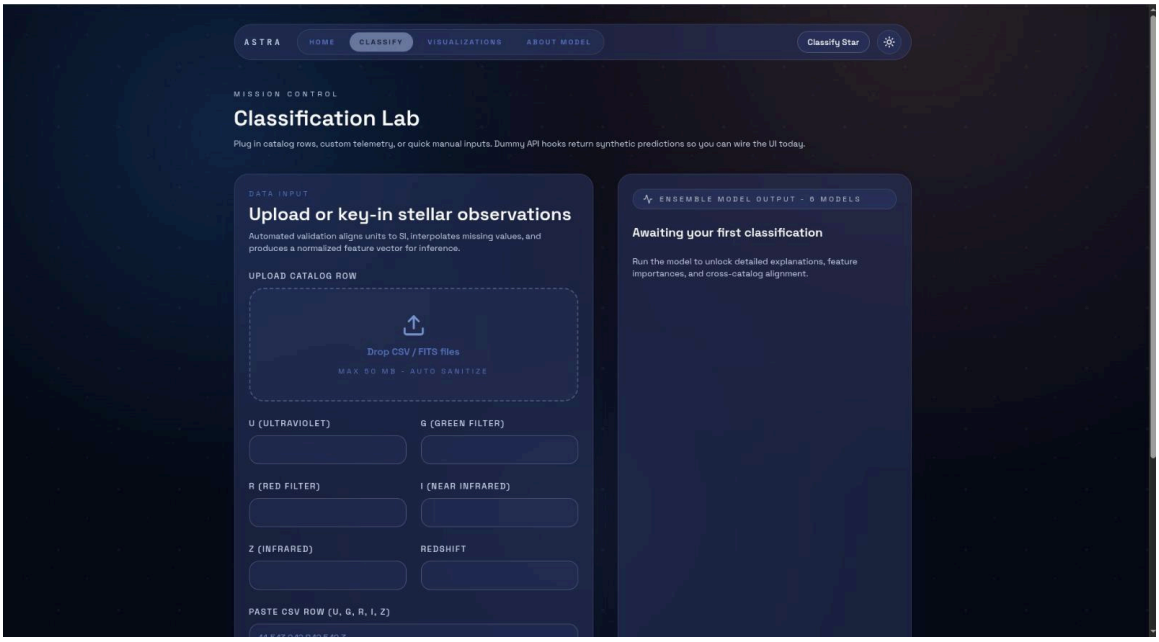
```
1. import numpy as np
2. import pandas as pd
3. import matplotlib.pyplot as plt
4. import seaborn as sns
5. from sklearn.linear_model import LogisticRegression
6. from sklearn.neighbors import KNeighborsClassifier
7. from sklearn.tree import DecisionTreeClassifier
8. from sklearn.naive_bayes import GaussianNB
9. from sklearn.ensemble import RandomForestClassifier
10. from sklearn.model_selection import train_test_split
11. from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
12. from sklearn.svm import SVC
13. from sklearn.model_selection import KFold, cross_val_score
```

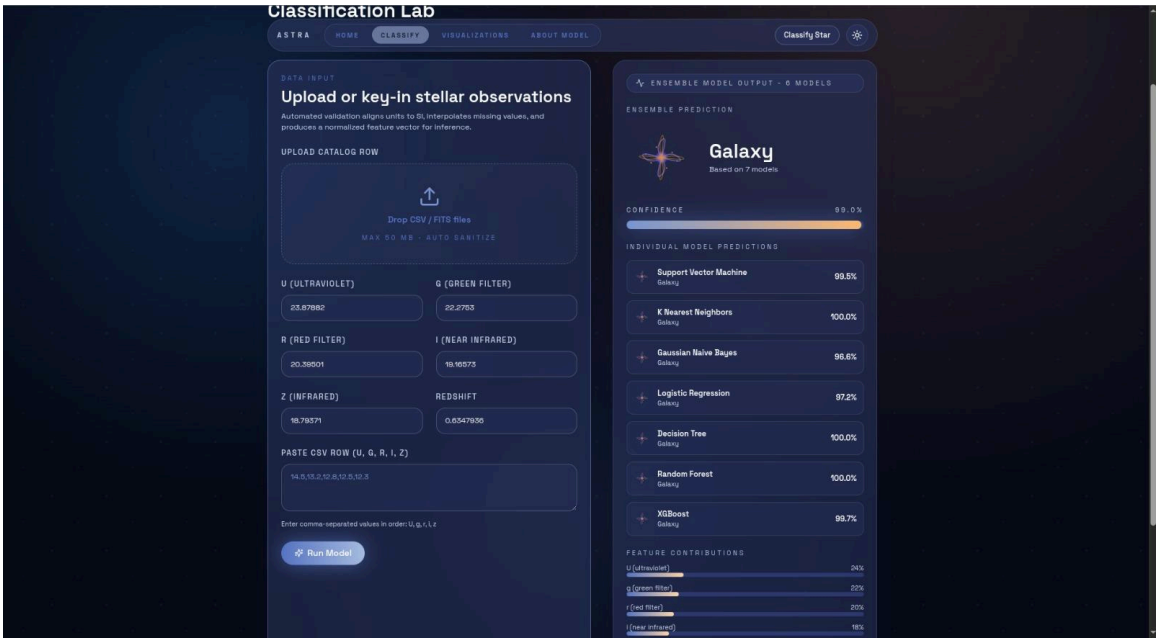
6. Design

6.1 UI/UX Design:

A highly interactive and visually driven user interface was developed to enable intuitive data input, transparent model interpretation, and engaging visualization of stellar classification outcomes was created using React.js and cursorAI..







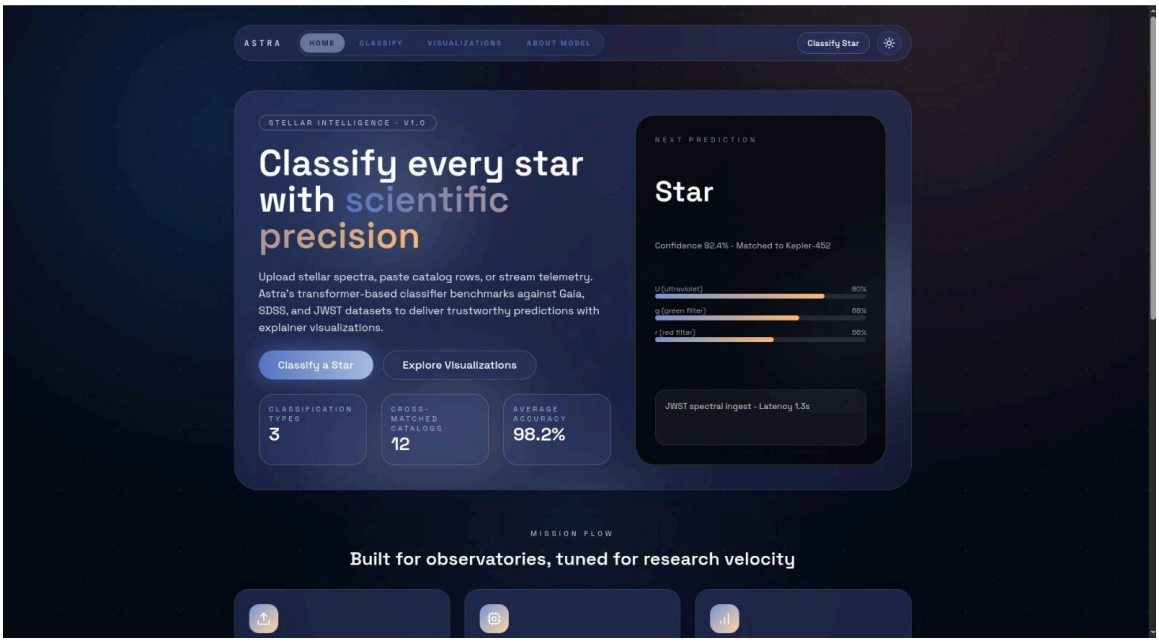


Figure21 - Figure24 : Are images of the site's UI UX

6.2 API / Integration Design

The API is built using Django REST Framework (DRF) and serves as the integration layer between the ML artifact registry and the user interface (UI/UX) developed using Cursor AI.

6.2.1 API Endpoint Design

Accepts numerical feature inputs and returns predictions from all ML models stored in the Model Registry

Request & Response Body Format:

```
{
  "u": <float>,
  "g": <float>,
  "r": <float>,
  "i": <float>,
  "z": <float>,
  "redshift": <float>
}
```

```
{
  "inputs": { ... },
  "predictions": {
      "svm": { "class": "GALAXY", "confidence": 0.9945 },
      "knn": { "class": "GALAXY", "confidence": 1.0 },
      ...
  },
  "meta": {
      "model_count": 7,
      "features": ["u", "g", "r", "i", "z", "redshift"]
  }
}
```

Figure25 & 26 : Shows the format of the response and request at endpoints

6.2.2 Data Flow

1. User Input (UI Layer)
    - The front-end interface created using Cursor AI captures the numeric parameters (u, g, r, i, z, redshift).
    - Sends the data to the Django backend using a JSON POST request.
2. API Input Validation
    - Django REST Framework's serializer validates the incoming fields for type, completeness, and constraints.
3. Model Registry Access
    - A centralized ModelRegistry loads all stored .pkl ML models and metadata (metadata.json) at server startup.
    - The registry provides quick retrieval of all models and the feature mapping.
4. Prediction Pipeline
    - Inputs are organized into a feature vector in the correct metadata-defined order.
    - Each ML model (SVM, KNN, Logistic Regression, Random Forest, Decision Tree, GaussianNB, XGBoost) performs:
    - .predict() to determine the class index.
    - .predict_proba() (if available) to compute confidence.
5. Label Mapping:

    Numerical predictions (0, 1, 2) are mapped to domain classes:

    - 0 → STAR
    - 1 → GALAXY
    - 2 → QSO
6. Response Construction
    - Model predictions and confidence scores are aggregated into a structured JSON response.
    - Metadata (feature names, model count) is appended.
    - Response is returned to the user interface.
7. Front-End Visualization
    - The UI displays predictions for all models, enabling comparison and transparency.

7. Implementation

Describe key components, modules, and noteworthy code segments.

7.1 Noteworthy code segment

Model Registry :

```
class ModelRegistry:
    models = {}
    metadata = {}

def load_models():
    with open(ARTIFACTS_DIR / "metadata.json") as f:
        ModelRegistry.metadata = json.load(f)

    for pkl in ARTIFACTS_DIR.glob("*.pkl"):
        ModelRegistry.models[pkl.stem] = joblib.load(pkl)
```

Figure27 : Shows the code-segment used in model registry

This ensured the API did not retrain models during every request, which saved us a lot of time because before we used it , the SVM and RandomForest models significantly increased response time by about 10 minutes

7.2 Key Component

The website not only provides prediction outputs but also offers detailed insights into the contribution of each feature to the final classification which helps users understand the importance of each feature,

Making it a fine tool for educators who prefer to go into minute details and impact while teaching.

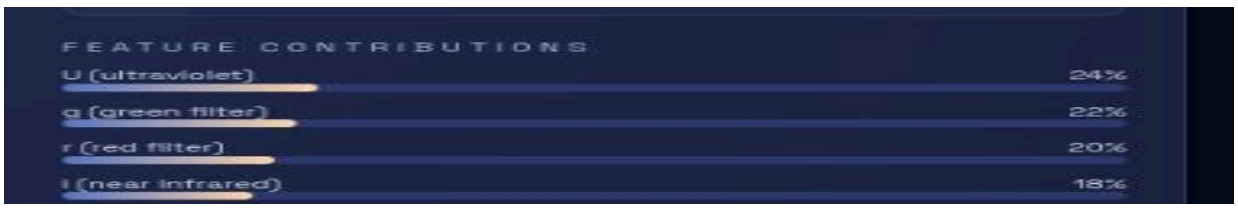


Figure28 : Shows an image taken from the site that shows feature contribution

8. Testing & Evaluation

8.1 Test Plan

8.1.1. Unit Testing

Unit testing was performed to validate the functionality of individual components in isolation.

1. ML Model Testing
    - Verified successful loading of trained model artifacts (.pkl files).
    - Tested prediction functions for correct input shape and output format.
    - Ensured class label mapping (STAR, GALAXY, QSO) was accurate.
2. Backend Components
    - Serializer validation for input features (u, g, r, i, z, redshift).
    - Model registry loading logic.
    - Predictor function handling multiple models simultaneously.

Tools Used: Python unittest, manual function testing.

8.1.2. Integration Testing

Integration testing ensured smooth interaction between machine learning models and the Django REST framework.

1. Verified end-to-end data flow:
    - Client request → Serializer → Model inference → API response
2. Checked compatibility between:
    - Django views and predictor logic
    - Metadata configuration and model outputs
3. Tested combined predictions from multiple models in a single API call.

Outcome: All trained models successfully returned predictions through the API.

8.1.3. System Testing

System testing validated the complete deployed application as a whole.

1. Tested REST API endpoints using:
    - Django REST Framework Browsable API
    - JSON POST requests

2. Verified response structure:
    - Input echo
    - Model-wise predictions
    - Confidence scores
    - Metadata details
3. Confirmed robustness against invalid or missing inputs.

Result: The system performed reliably under expected operating conditions.

8.1.4.User Testing

User testing focused on usability and practical interaction.

1. UI/UX interface (developed using Cursor AI) tested for:
    - Ease of data entry
    - Correct API connectivity
    - Real-time display of prediction results
2. Outputs collected regarding clarity of results and response speed.

Observation: The system was intuitive and provided clear, interpretable outputs to users.

8.2 Test Cases & Results: Summaries and defect fixes.

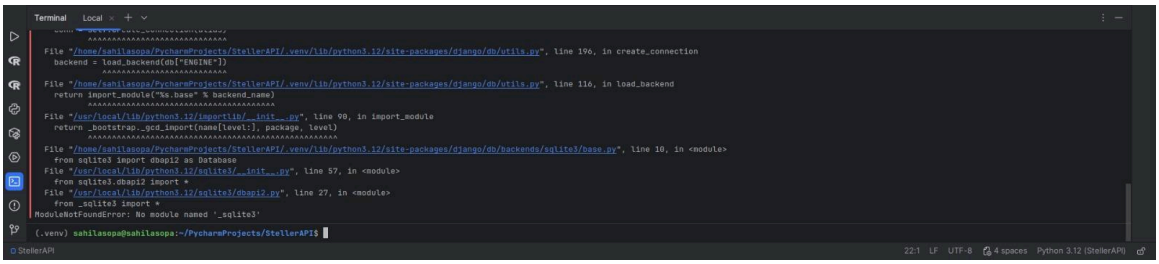


Figure29 : Shows our most troublesome code error we encountered

Even though Django uses SQLite by default, our Python 3.12 installation from the system packages did not compile with SQLite support because:

- Required packages (libsqlite3-dev, sqlite3) were missing at build time.
- The virtual environment inherited this broken Python build.

As a result, the internal Python module _sqlite3 was unavailable, causing the runtime import error.
We fixed the issue by switching to a Conda environment, which bundles SQLite support by default.

After migrating to a Conda environment:

- The _sqlite3 module imported successfully.
- Django's ORM worked normally.
- Our prediction API loaded models without errors.
- Backend development continued smoothly.

Another noteworthy challenge we encountered was a CUDA compatibility issue which arose when using the latest XGBoost release, resulting in instability during GPU-accelerated training. To resolve this, a stable and CUDA-compatible XGBoost version (v2.0.0) was adopted, ensuring reliable execution and consistent performance.

8.3 Performance Metrics:

The following performance metrics were used for all the models :-

- Accuracy
- F1-Score
- Precision
- Recall
- K-Fold Cross-Validation

By finding these metrics we concluded Random Forest and XGBoost to be the best or top performing models compared to the rest .

9. Results

1]Logistics regression :

```
...   Logistic Regression
    Accuracy: 0.93245
    Classification Report:
                  precision    recall  f1-score   support

               0       0.90      1.00      0.95      4319
               1       0.97      0.91      0.94     11889
               2       0.85      0.93      0.89      3792

        accuracy                           0.93     20000
       macro avg       0.91      0.95      0.93     20000
    weighted avg       0.94      0.93      0.93     20000

    Confusion Matrix:
     [[ 4315     4     0]
     [  458 10823   608]
     [    2   279  3511]]
```

Figure30 : Shows logistic regression results pre-cross validation

2]Decision Tree

```
Decision Tree
Accuracy: 0.96705
Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.99      1.00      4319
           1       0.97      0.97      0.97     11889
           2       0.92      0.92      0.92      3792

    accuracy                           0.97     20000
   macro avg       0.96      0.96      0.96     20000
weighted avg       0.97      0.97      0.97     20000

Confusion Matrix:
 [[ 4297    22     0]
 [   18 11553   318]
 [    0   301  3491]]
```

Figure31: Shows Decision Tree results pre-cross validation

3]Random Forest

```
 Random Forest
Accuracy: 0.97995
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      4319
           1       0.98      0.99      0.98     11889
           2       0.97      0.93      0.95      3792

    accuracy                           0.98     20000
   macro avg       0.98      0.97      0.98     20000
weighted avg       0.98      0.98      0.98     20000

Confusion Matrix:
 [[ 4316     3     0]
 [   17 11745   127]
 [    0   254  3538]]
```

Figure32 : Shows random forest results pre-cross validation

4]KNN

```
 KNN
Accuracy: 0.94975
Classification Report:
              precision    recall  f1-score   support

           0       0.92      0.96      0.94      4319
           1       0.96      0.96      0.96     11889
           2       0.96      0.92      0.94      3792

    accuracy                           0.95     20000
   macro avg       0.94      0.94      0.94     20000
weighted avg       0.95      0.95      0.95     20000

Confusion Matrix:
 [[ 4134   177     8]
 [  365 11373   151]
 [   18   286  3488]]
```

Figure33 : Shows KNN results pre-cross validation

5]Support Vector Machine

```
 Model Evaluation:
Accuracy: 0.7755
Classification Report:              precision    recall  f1-score   support

           0       0.55      0.93      0.69      4319
           1       0.94      0.67      0.79     11889
           2       0.84      0.91      0.87      3792

    accuracy                           0.78     20000
   macro avg       0.78      0.84      0.78     20000
weighted avg       0.84      0.78      0.78     20000
```

Figure34 : Shows SVM results pre-cross validation

6]Gaussian Naives Bayes

```
 Gaussian Naive Bayes
Accuracy: 0.7355
Classification Report:
              precision    recall  f1-score   support

           0       0.99      0.12      0.22      4319
           1       0.75      0.91      0.82     11889
           2       0.67      0.89      0.76      3792

    accuracy                           0.74     20000
   macro avg       0.80      0.64      0.60     20000
weighted avg       0.79      0.74      0.68     20000

Confusion Matrix:
 [[  524  3182   613]
 [    7 10827  1055]
 [    0   433  3359]]
```

Figure35 : Shows Gaussian Naives Bayes results pre-cross validation

7]XGBoost

```
 XGBoost
Accuracy: 0.9763
Classification Report:
              precision    recall  f1-score   support

           0       0.99      0.99      0.99      4319
           1       0.98      0.98      0.98     11889
           2       0.96      0.93      0.95      3792

    accuracy                           0.98     20000
   macro avg       0.98      0.97      0.97     20000
weighted avg       0.98      0.98      0.98     20000

Confusion Matrix:
 [[ 4287    32     0]
 [   42 11703   144]
 [    1   255  3536]]
```

Figure37 : Shows XGBoost results pre-cross validation

8]Cross validation of all models

```
Logistic Regression K-Fold Accuracy:
Mean: 0.9351299999999998
Std : 0.0015078461459976417

KNN K-Fold Accuracy:
Mean: 0.95006
Std : 0.0015954309762568747

Decision Tree K-Fold Accuracy:
Mean: 0.96602
Std : 0.0012048236385463396

Gaussian NB K-Fold Accuracy:
Mean: 0.77265
Std : 0.07429612372122789

Random Forest K-Fold Accuracy:
Mean: 0.97876
Std : 0.0013897481786280539

XGBoost K-Fold Accuracy:
Mean: 0.97575
Std : 0.0012501999840025505
 K-Fold Cross-Validation Results
Logistic Regression: 0.9351299999999998
KNN: 0.95006
Decision Tree: 0.96602
Gaussian NB: 0.77265
Random Forest: 0.97876
XGBoost: 0.97575
```

Figure38 : Shows cross validation results of each model along with their mean and standard deviation

10. Conclusion & Future Work

This project successfully developed an end-to-end machine learning system for stellar object classification, integrating data preprocessing, multiple classification models, rigorous evaluation using K-Fold cross-validation, and an interactive web interface. Among the evaluated models, ensemble-based approaches such as Random Forest and XGBoost demonstrated superior performance in terms of accuracy and stability. The system not only provides reliable predictions but also emphasizes model interpretability by presenting feature-level contributions, enhancing transparency and user trust.

As future work, the project is planned to be expanded to cloud-based infrastructure using **Amazon AWS EC2** for scalable hosting and **Amazon Bedrock** for integrating advanced foundation models and inference capabilities. Additionally, the system aims to incorporate **continuous and real-time astronomical datasets**, which will significantly improve model robustness and real-world applicability. However, the acquisition and processing of such datasets involve substantial financial and computational costs, which limit immediate implementation. Addressing these constraints through cloud scalability and optimized data pipelines forms a key direction for future development.

11. Budget & Resources

At present, the system is deployed on a Raspberry Pi to minimize costs and uses a completely free Kaggle Dataset. However for scalability and higher availability, the project may be migrated to Amazon AWS, incurring an estimated cost of $20 per month.

References

References

[1]S. Chaini et al., "Photometric identification of compact galaxies, stars, and quasars," Mon. Not. R. Astron. Soc., vol. 518, no. 2, pp. 3123-3138, 2022

[2]Y. Li et al., "Using CFSVM model to classify stars from three-colour photometry," in Proc. IEEE Int. Conf. Big Data, 2021, doi: 10.1109/BigData52589.2021.9671583. [web:18]

Appendices

User Manual:

Step 1: Open the web application through a supported browser.

Step 2: Locate the input form on the user interface.

Step 3: Enter the required astronomical parameters:

- u, g, r, i, z magnitudes ,Redshift

Step 4: Verify that all inputs are numeric and valid.

Step 5: Click the Predict button to submit the data.

Step 6: The frontend sends the input data to the Django REST API.

Step 7: The backend processes the request using pre-trained machine learning models.

Step 8: The system returns classification results for each model.

Step 9: View the predicted class labels (STAR / GALAXY / QSO) along with confidence scores.

Step 10: Use the results for analysis or decision-making.

GITHUB REPOSITORY :

https://github.com/sahilasopa/stellerapi

https://github.com/sahilasopa/stellerui